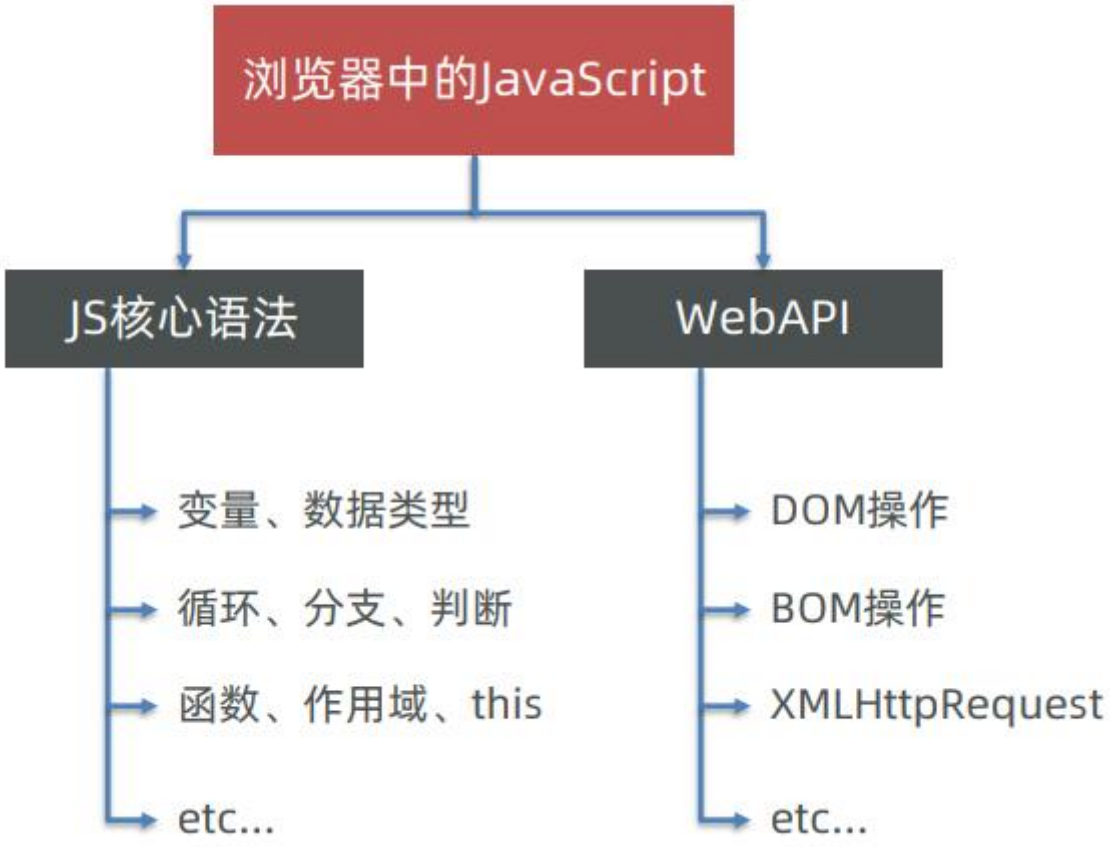
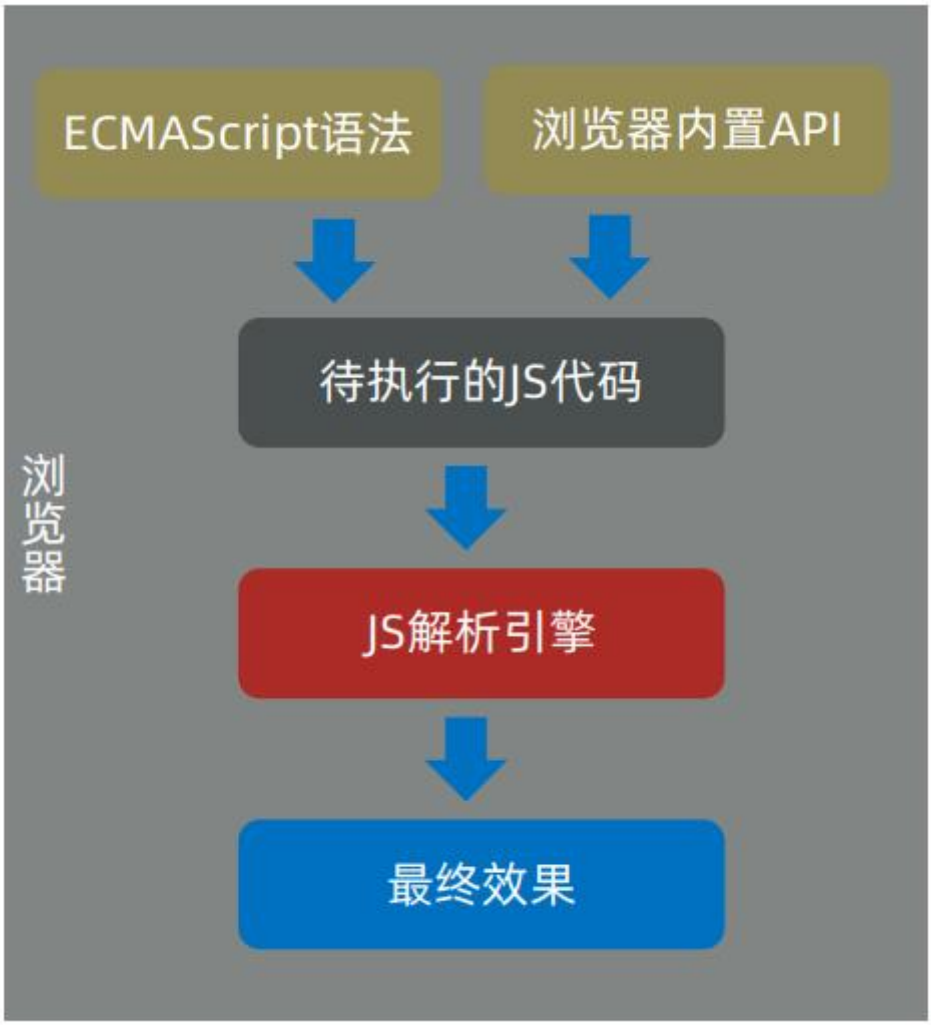
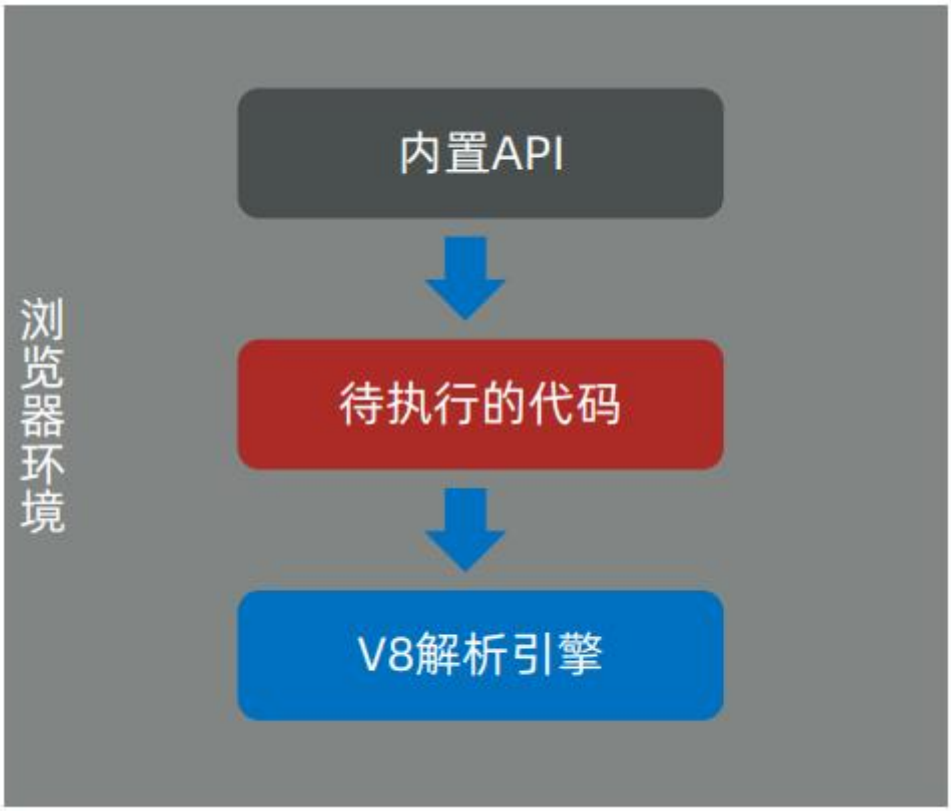
- **浏览器中的JavaScript组成**
  - 组成部分
    - JS核心语法（ECMAScript）
      - 变量、数据类型
      - 循环、分支、判断
      - 函数、作用域、this
      - etc...
    - WebAPI
      - DOM
      - BOM
      - XMLHttpRequest
      - canvas
      - etc...



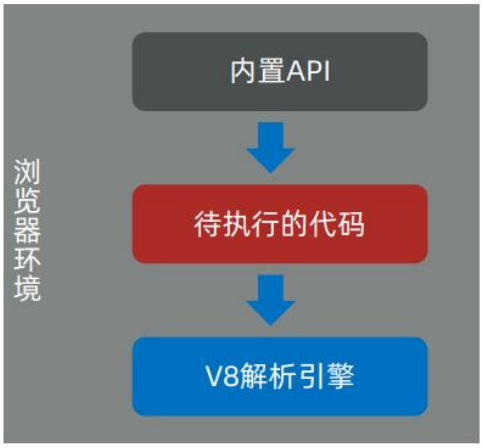
  - JavaScript代码是如何被转换成最终效果的
    - 不同的浏览器使用不同的 JavaScript 解析引擎
      - Chrome 浏览器 => V8
      - Firefox 浏览器=> OdinMonkey(奥丁猴)
      - Safri 浏览器=> JSCore
      - IE 浏览器=> Chakra(查克拉)
      - etc...
    - 其中，Chrome 浏览器的 V8 解析引擎性能最好

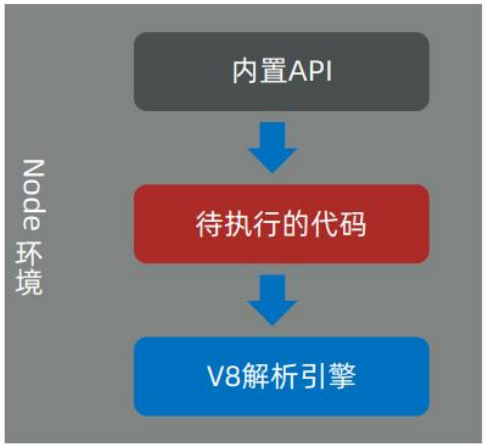


- JavaScript 运行环境
  - 环境，和生活中的环境一样，比如人生存的环境
    - 必须有水
    - 必须有氧气
    - 必须有女朋友
    - ……
  - 运行环境指的是代码正常运行所需的必要条件。
    - 必须有内置API （才能写代码）
    - 必须有解析引擎（才能运行代码）



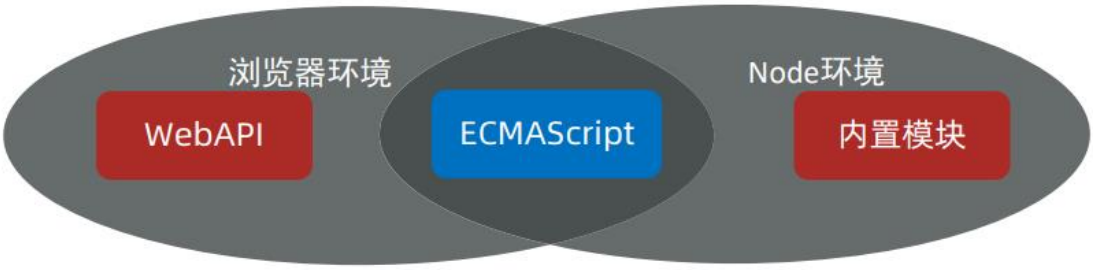
- **Node.js 介绍**
  - Node.js可以做什么
    - 基于 Express/Koa 框架(http://www.expressjs.com.cn/)，可以快速构建 Web 应用
    - 基于 Electron 框架(https://electronjs.org/)， 可以构建跨平台的桌面应用，比如 vscode
    - 基于 restify 框架(http://restify.com/)，可以快速构建 API 接口项目
    - 读写和操作数据库、创建实用的命令行工具辅助前端开发
  - Node 环境
    - Node 是一个基于 Chrome V8 引擎的 JavaScript 运行环境。
    - 通俗的理解：Node 为 Node.js 代码的正常运行，提供的必要的环境。





  - 总结：
    - 浏览器 和 Node，都是 JS 的运行环境
    - 具体来说，浏览器是客户端的JS（JavaScript ）环境；Node是服务端的JS（Node.js）运行环境
    - 不同环境中，都可以运行 ECMAScript 核心代码

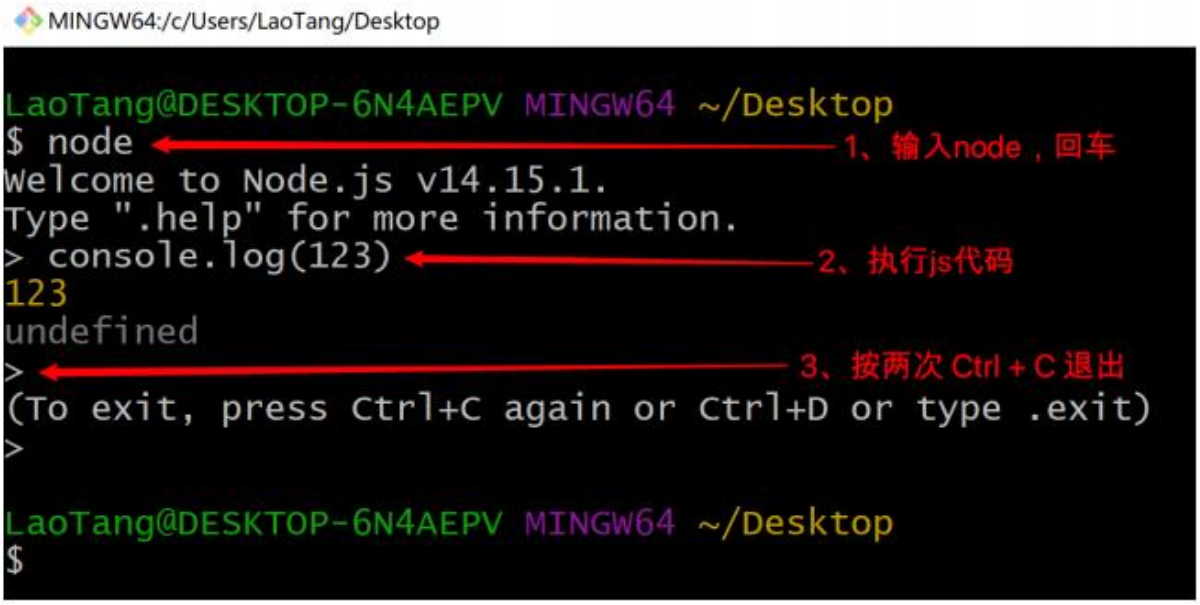
    - WebAPI是浏览器特有的，只能在浏览器环境下使用；
    - Node内置API（内置模块）是Node环境特有的，只能在Node环境中使用



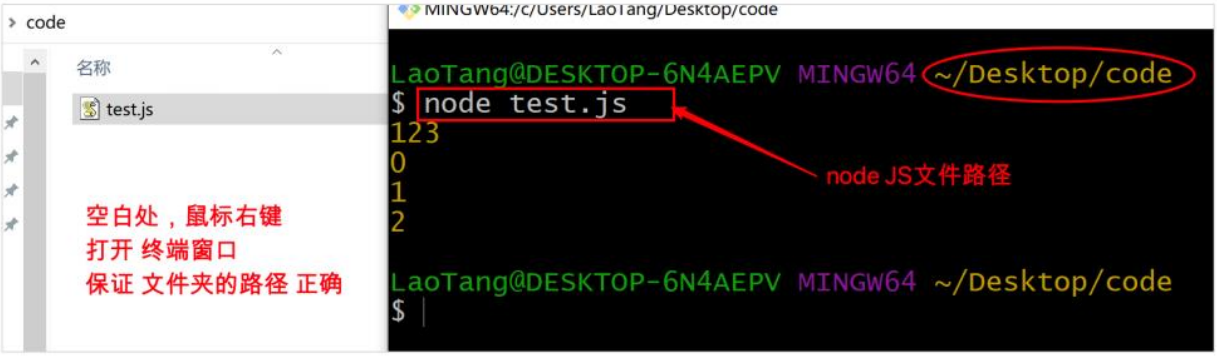
- **Node.js 安装**
    - 获取Node.js
        - 安装包可以从 Node.js 的官网首页直接下载，进入到 Node.js 的官网首页，点 击绿色的按钮，下载所需的版本后，双击直接安装即可。

        - 长期支持版，基本没用重大bug，推荐大多数用户使用
        - 当前发布版，含最新功能，但可能存在未发现的重大bug
    - 安装Node.js
        - 打开下载好的安装包，一直下一步即可安装。
        - 如果不想安装在C盘，则把安装路径中的C修改为D或者E即可。
        - 安装后，打开终端窗口，执行 node -v 命令，如果看到版本号，说明安装成功。

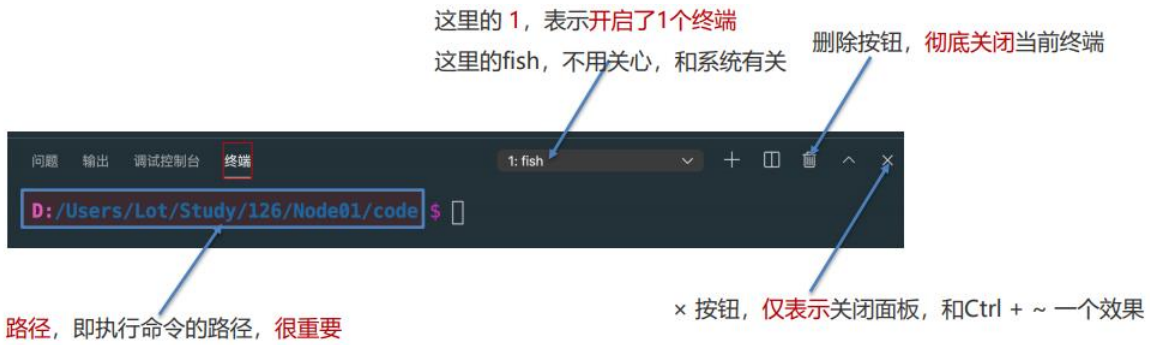
- **Node环境中运行JavaScript**
    - 交互模式
        - REPL(Read-Eval -Print-Loop)交互模式
        - 指的是在终端窗口中，执行简单的JavaScript代码
        - 操作步骤：
            - 打开任意终端，直接输入 node 命令并回车
            - 执行你的JS代码，按回车表示执行
            - 按两次“Ctrl+C”退出



- 脚本模式
  - 如果有大段的JS代码需要在Node环境中运行
  - 可以把JS代码写到JS文件中
  - 终端中，使用 node xxx.js 命令即可运行文件中的代码

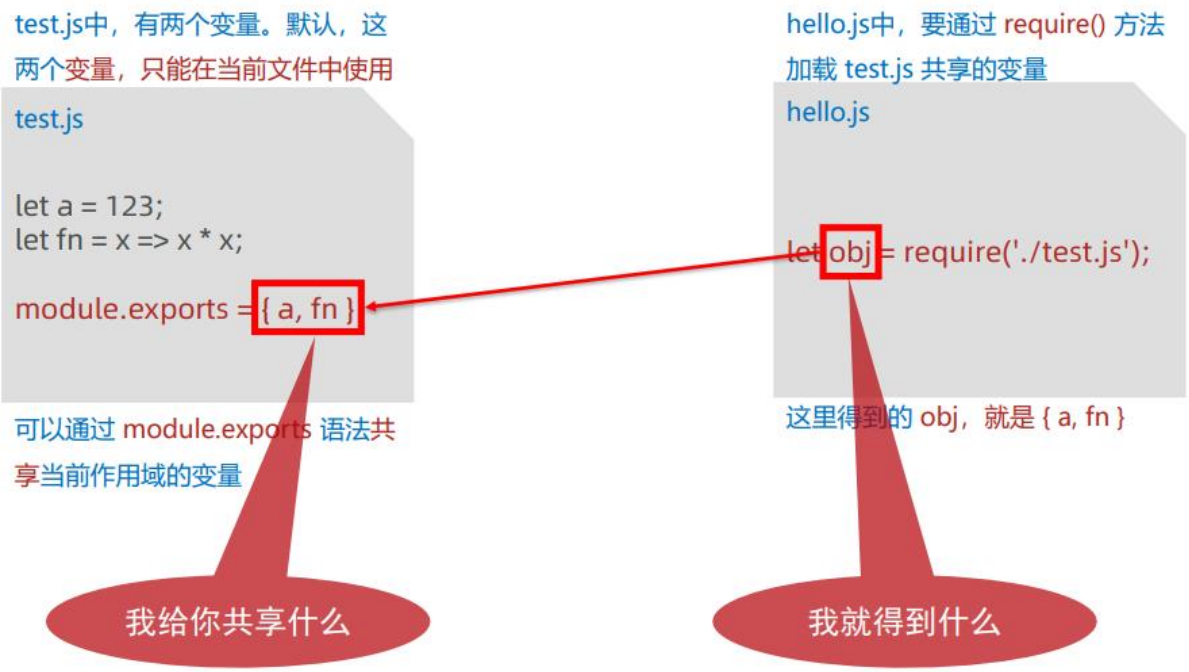


- **vscode中的终端**
  - 无论执行 git 命令，还是 node 命令，都需要使用终端工具
  - vscode内置了终端工具，我们可以使用它来执行 git 或 node 命令
  - 最佳打开vscode终端的方式
    - 在文件或文件夹上，鼠标右键，在集成终端中打开
    - 这样打开的好处是，终端路径刚好是 当前文件或文件夹的路径



- **模块化设计**

- Node中的模块化
  - 规定，用户创建的每个JS文件，就是一个小模块，叫做自定义模块
  - 我们可以按照一定的语法将这些小模块组合到一起，形成一个完整的项目
  - Node中的模块化，通俗的说，就是在JS文件中，能够使用另一个文件中的变量
  - 言外之意，就是把两个毫不相干的JS文件（零件）组装到一起
- Node中模块化的实现



- Node中的模块分类
  - 自定义模块：用户自己创建的每个JS文件，都是自定义模块
  - 内置模块（核心模块）：Node安装后，即可使用的模块，Node环境自带
  - 第三方模块：其他人或公司、组织开发的模块，发布到 npm 网站，我们需要下载使用的模块
- 加载模块的语法
  - 加载自定义模块
    - a 文件，必须通过 module.exports 共享（导出、暴露）当前模块中的变量
    - b 文件，需要通过 require() 加载（导入）。（a 文件导出什么，b 文件就得到什么）
    - 加载自定义模块时，必须要带路径 （相对路径、绝对路径都可以；但必须带路径，哪怕是 ./ 也不能省略）
    - 加载自定义模块时，可以省略后缀
  - 加载内置模块
    - 直接使用 require() 加载即可。
    - 加载模块时，无需带路径，比如 require('fs')
    - 加载内置模块，得到对象类型，对象中内置了很多API方法
  - 加载第三方模块
    - 必须先下载（后续讲解）

- **内置模块介绍**

- 内置模块，顾名思义，即 Node 环境自带的模块；安装完Node即可直接使用
- Node内置模块有很多，具体参见 http://nodejs.cn/api/
- 加载内置模块：let 变量 = require('模块名');
- 模块和模块之间很好区分，看模块名即可确定他们的作用

- **内置模块 - fs 模块**
  - fs 模块，f（file）s（system），文件系统；所有关于文件的操作，都可以通过这个模块完成
    - 比如创建文件
    - 获取文件里面的内容
    - 向文件中添加内容
    - 创建文件夹
    - 遍历文件夹里面的文件
    - 监视文件的变化
    - 判断文件是否存在
    - ……
  - fs模块 - fs.readFile()
    - fs.readFile() 方法的作用是：读取文件
      - 读取：获取
      - 读取文件：获取文件里面的内容
      - 注意，这是一个异步方法

```
fs.readFile('文件路径', [utf-8], (err, data) => {
  // 读取无错误，err=null; 读取有错误，err=错误对象
  // data 表示读取的结果
})
```

  - fs模块 - fs.writeFile()
    - fs.writeFile() 方法的作用是：写入文件
      - 写入：向文件中添加内容
      - 特点：如果文件不存在，则会创建文件（但不能递归创建）
      - 特点：如果文件中有内容，将会被覆盖
      - 注意，这是一个异步方法

```
fs.writeFile('文件路径', '写入的内容', err => {
  // 写入无错误，err=null; 写入有错误，err=错误对象
})
```

- fs模块 - fs.access()
  - fs.access() 方法的作用是：判断文件是否存在（是否可读、是否可写）
    - 参数2可选；
      - fs.constants.F_OK或不填，表示判断文件是否存在；
      - fs.constants.R_OK表示判断文件是否可读；
      - fs.constants.W_OK表示文件是否可写
    - 注意，这是一个异步方法

```
fs.access('文件路径', [选填], err => {
  // 文件存在，err=null；不存在err=错误对象
})
```

- **内置模块 - path 模块**
  - path 模块，path 是路径的意思；所有和路径相关的操作，都可以通过这个模块完成
    - 拼接一个路径
    - 获取当前文件所在的路径
    - 获取路径中的文件名
    - 获取路径中的后缀
    - ……
  - path模块 - path.extname()
    - path.extname() 方法的作用是：获取路径中的后缀
      - 参数：一个路径

```
path.extname('文件路径');

path.extname('css/index.css'); // .css

path.extname('E:/a/b/index.html'); // .html

path.extname('assets/js/login.xyz'); // xyz

path.extname('abc/bcd/index'); // null
```

  - path模块 - path.join()
    - path.join() 方法的作用是：拼接给出的路径
      - 参数：两个或更多个路径
      - 额外补充：Node中，有一个全局变量 __dirname ，它表示当前 JS 文件所在的绝对路径

```
console.log(__dirname); // D:/study/100/Node01/code

path.join('路径', '路径', ……);

path.join('css', 'index.css'); // css/index.css

path.join('a', 'b', 'c', 'd.js'); // a/b/c/d.js

path.extname(__dirname,  'js', 'a.js');
// D:/study/100/Node01/code/js/a.js
```

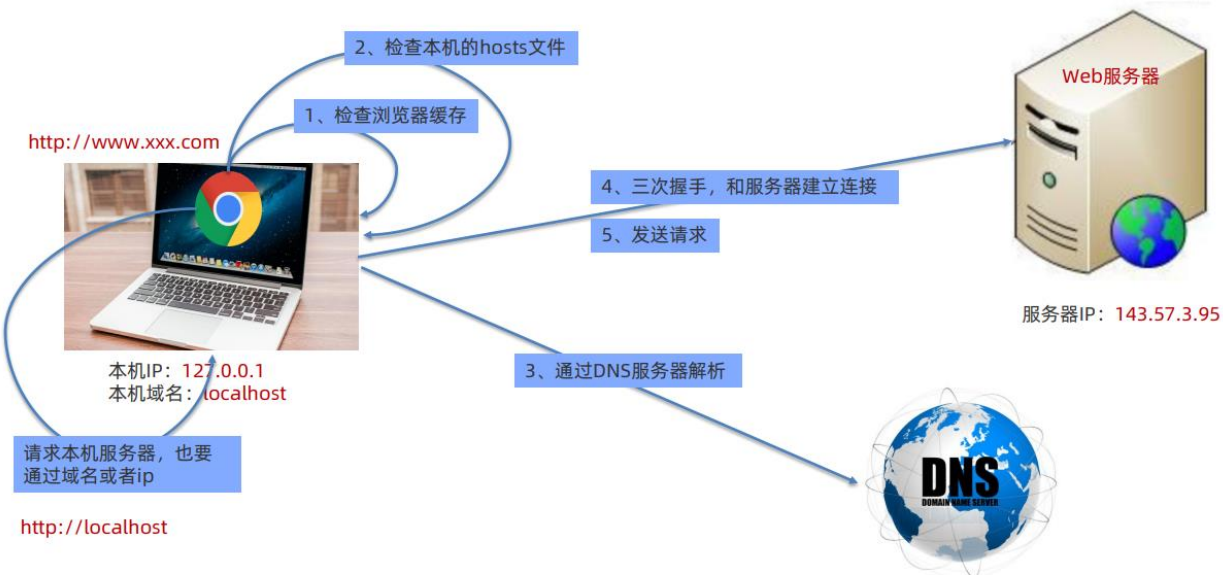
- **内置模块 - querystring 模块**
  - querystring 模块，querystring 是查询字符串的意思；所有和查询字符串相关的操作，都可以通过这个模块完成
    - 什么格式的字符串是查询字符串：id=1&name=zs&age=20
    - 将查询字符串，转换成对象
    - 将对象转成查询字符串
    - ……
  - querystring模块 - querystring.parse()
    - querystring.parse() 方法的作用是：将查询字符串转换成JS对象
      - 参数：一个查询字符串

```
querystring.parse('id=1&name=zs&age=20');

// { id: 1, name: 'zs', age: 20 }
```

  - querystring模块 - querystring.stringify()
    - querystring.stringify() 方法的作用是：将JS对象 转成 查询字符串
      - 参数：一个字面量JS对象

```
querystring.stringify({ id: 1, name: 'zs', age: 2 });

// id=1&name=zs&age=2
```

- **http模块**
  - 请求一个网站的基本流程



- http模块介绍
  - http模块，也是Node的内置模块，也是通过 require('http') 加载使用
  - http模块，是和网络请求相关的模块
  - http模块，可以搭建web服务器，可以向其他服务器发送http请求
- 搭建web服务器的步骤
  - 加载http模块

```
const http = require('http');
```

  - 创建 server 对象

```
const server = http.createServer();
```

  - 注册 request 事件

```
server.on('request', (req, res) => {
    res.setHeader('Content-Type', 'text/plain; charset=utf-8');
    res.end('你好世界');
})
```

  - 指定端口，启动服务

```
server.listen(4000, () => console.log('服务器启动了'))
```

- 步骤解析
  - 第 ① 步：写法固定
  - 第 ② 步：写法基本固定
  - 第 ③ 步：用于处理客户端的请求；
    - 这一步，可以和第 ② 步合并到一起
    - 当客户端发送请求到达服务器的时候，就会触发 request 事件，执行 (req, res) => {} 函数
    - 通过 req （request）对象，可以获取到请求相关的信息
    - 通过 res （response）对象，可以做出响应处理
  - 第 ④ 步：写法基本固定
    - 端口 4000 可以修改，只要不和其他程序的端口冲突即可
- res对象
  - 这里指的 res ，是处理函数的第二个形参

```
server.on('request', (req, res) => {
    // 取自单词 response，响应的意思
});
```

  - 处理响应信息，都可以通过它完成
  - res.statusCode = 200； 可以设置响应状态码
  - res.setHeader('key', 'value'); 可以设置响应头
    - 如果响应中文，必须设置 res.setHeader('Content-Type', 'xxx/xxx; charset=utf-8'); 否则乱码
  - res.end(响应体); 设置响应体，并做出响应
    - 这个方法，必须放到最后；做出响应后，不能再设置响应状态码和响应头了，也不能多次调用 res.end() 方法
    - 响应体，只能是 字符串，如果要响应对象必须先转成字符串，即便是数字也必须加引号，比如 '123'
- req对象
  - 这里指的 req ，是处理函数的第一个形参

```
server.on('request', (req, res) => {
    // 取自单词 request，请求的意思
});
```

  - 所有请求相关信息，都可以通过它获取到
  - req.method 获取请求方式，比如得到 GET 、POST等等
  - req.url 获取请求地址中的除根路径以为的部分
    - http://localhost:3006 ===> req.url = '/'
    - http://localhost:3006/api/getbooks ===> req.url = '/api/getbooks'

        - http://localhost:3006/index.html ===> req.url = '/index.html'
        - http://localhost:3006/index.html?id=1 ===> req.url = '/index.html?id=1'
    - 搭建静态服务器

```
server.on('request', (req, res) => {
    // 获取请求路径，如果请求路径 为 / ，也读取 /index.html
    if (req.url === '/') req.url = '/index.html';
    // 根据请求路径判断文件是否存在
    fs.access('.' + req.url, err => {
        // 根据err判断文件是否存在
        if (err) {
            // 响应404
        } else {
            // 读取文件，响应
        }
    })
});
```